



Administrator Linux.Professional

Docker: Volumes and Networks



Проверить, идет ли запись

Меня хорошо видно && слышно?



Ставим "+", если все хорошо
"-", если есть проблемы





Шиков Станислав

Начальник отдела автоматизации ООО Кодер

Об опыте:

9+ лет в DevOps

начинал с системного администрирования

преподаватель в OTUS на курсах DevOps, Linux, SRE и некоторых других, автор нескольких курсов

Эл. почта:

stenlav@mail.ru



Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в ТГ-группе



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом



Документ



Ответьте себе или
задайте вопрос

Маршрут вебинара

Volumes and Networks

Закрепление навыков работы с Docker CLI

Настройка собственного Docker-registry

Рефлексия



Цели вебинара

После занятия вы сможете

1. Работать с docker networks и docker volumes
2. Понять основные принципы работы Docker registry
3. Применить на практике навыки управления Docker



Смысл

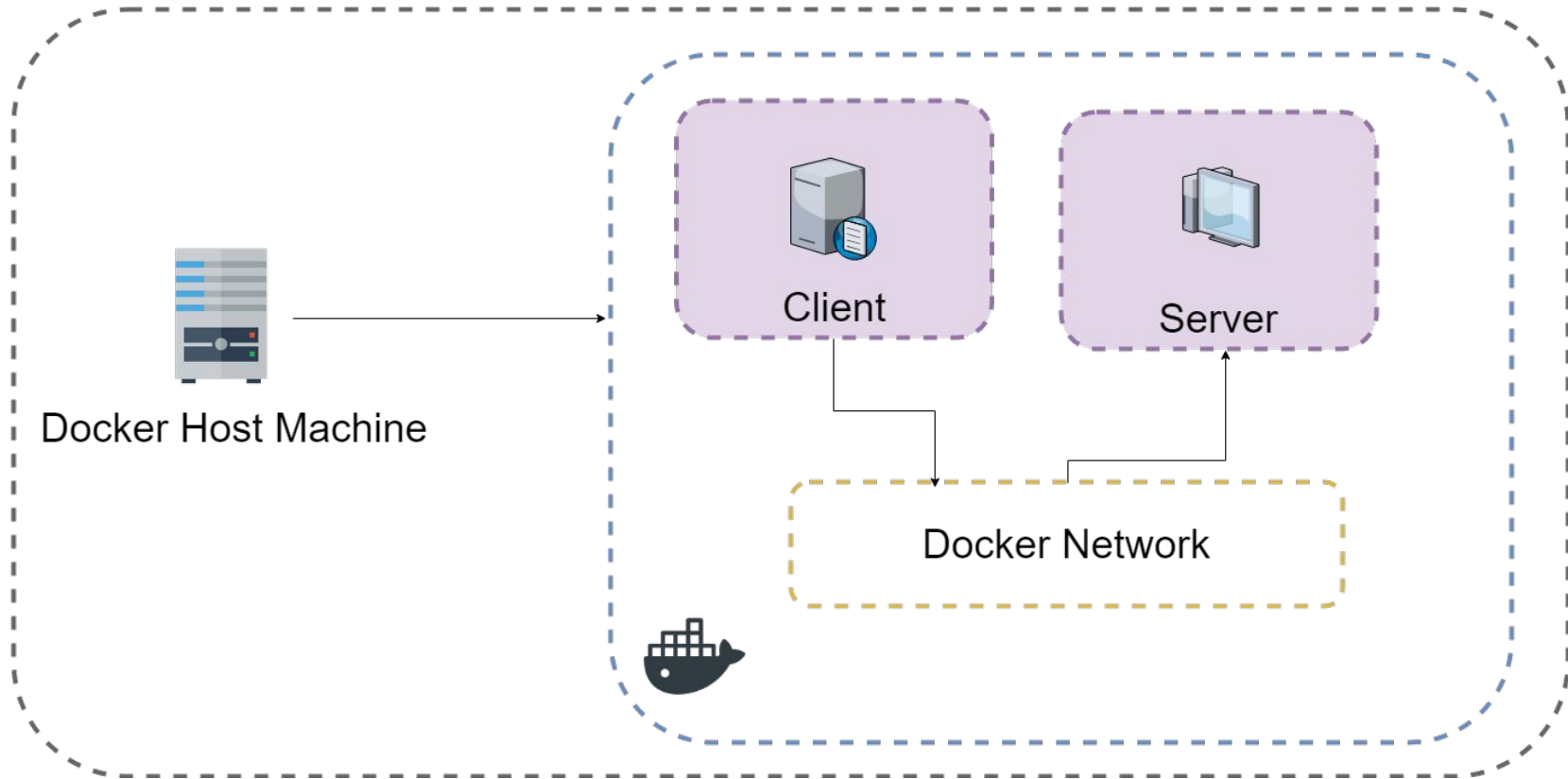
Зачем вам это уметь

1. Docker registry — основа работы с контейнеризацией в CI/CD
2. Docker Volumes and Networks — самые частые вопросы на собеседованиях
3. Docker = Easy to learn hard to master

Сети в Docker (Networks)



Docker Networks



Типы сетевых драйверов Docker

- **Bridge**
Изолированная сеть на одном хосте (*основной вариант*)
- **Host**
Общий сетевой стек с хостом
- **None**
Сеть отключена
- **Overlay**
Сеть между Docker-хостами (*Swarm*)
- **Macvlan**
Прямое подключение к физической сети
- **IPvlan**
Полный контроль над IPv4 и IPv6 адресацией

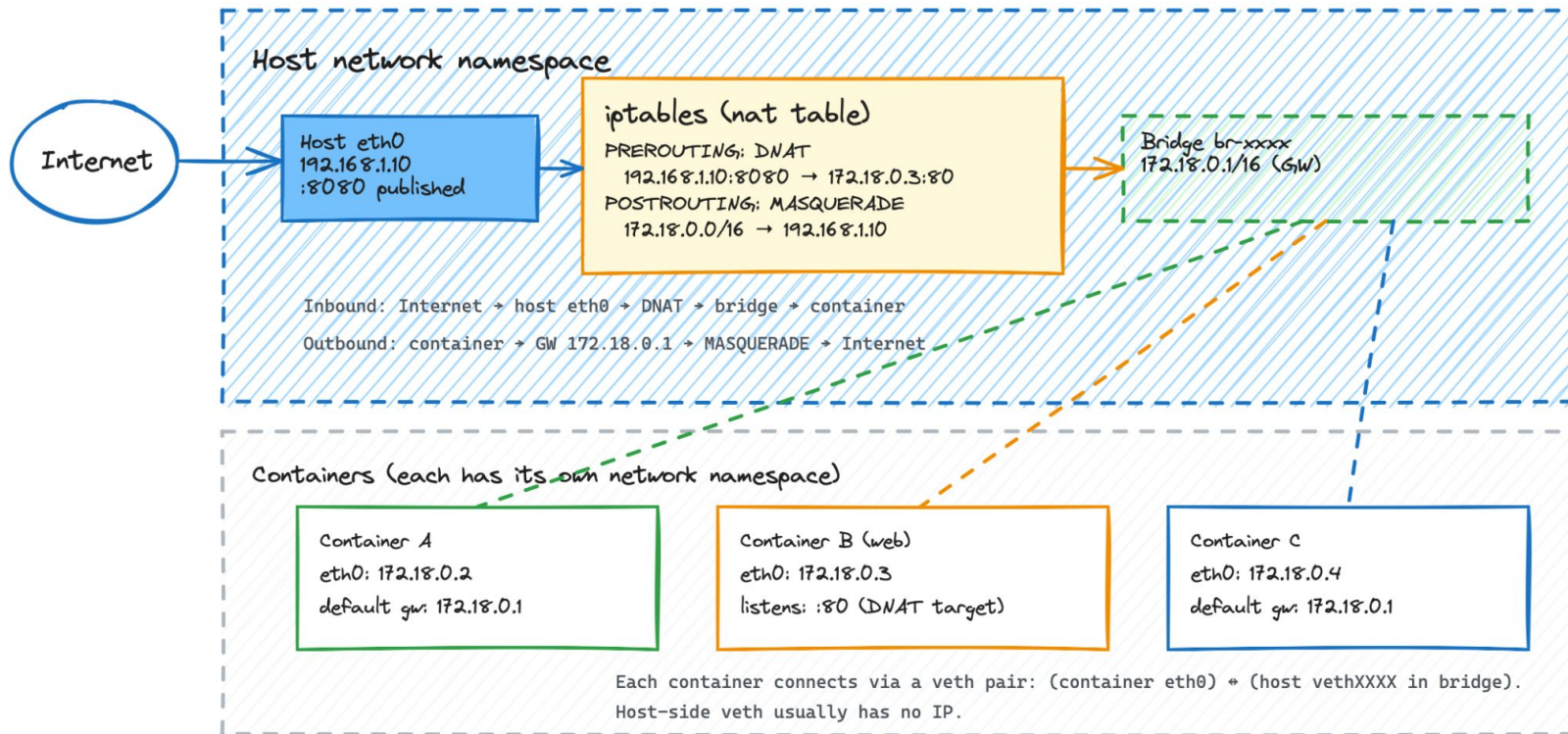
Bridge

- Изолированная сеть на одном Docker-хосте
- Основана на Linux bridge (**docker0**)
- Поддерживает проброс портов (**-p**)
- Контейнеры могут общаться между собой

Рекомендуется использовать user-defined bridge

Bridge

Docker Bridge Network (user-defined)



Работа с дефолтной bridge-сетью в Docker

```
docker run -d --name nginx -p 80:80 nginx
```

```
docker run -d --name nginx -p 127.0.0.1:80:80 nginx
```

```
docker run -d --name nginx -p 192.168.0.1:80:80 nginx
```

```
docker network ls
```

<https://docs.docker.com/network/#published-ports>

Работа с кастомной сетью в Docker

```
docker run -dt --name k1 nginx:alpine
```

```
docker run -dt --name k2 nginx:alpine
```

```
docker exec -it k1 sh -c "ip a"
```

```
docker exec -it k2 sh -c "ip a"
```

```
docker network ls
```

```
docker network create net
```

```
docker network create --driver bridge net2
```

```
docker run -dt --name n1 --network net nginx:alpine
```

```
docker run -dt --name n2 --network net nginx:alpine
```

```
docker run -it -d --network host --name nginx nginx
```

```
docker network inspect net
```

Host

- Контейнер использует **сетевой стек хоста**
- Нет сетевой изоляции
- Нет NAT и проброса портов
- Возможны конфликты портов

Когда применять:

Осознанно, для высокопроизводительных сервисов или диагностики

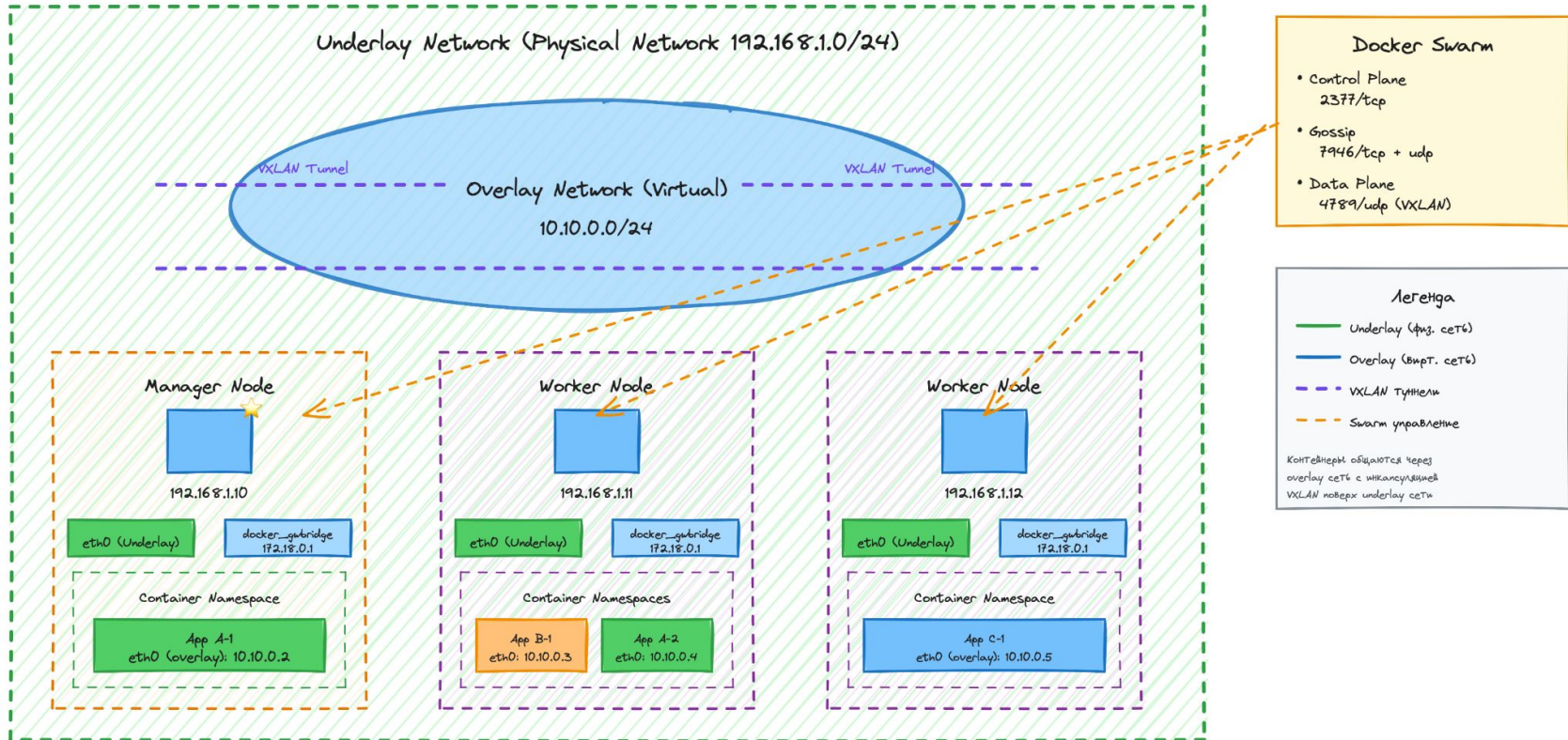


Повышенный риск для безопасности

Overlay

- Сеть между контейнерами на разных хостах
- Работает поверх VXLAN
- Требуется Docker Swarm

Overlay



* Overlay сеть (10.10.0.0/24) – виртуальная сеть, связывающая контейнеры на разных хостах

* VXLAN туннели автоматически создаются между хостами для передачи overlay трафика

Работа с Docker Swarm

```
docker swarm init
```

```
docker swarm join --token ...
```

```
docker network ls
```

```
docker network create --driver overlay --subnet 10.10.10.0/24 --attachable  
lab-overlay
```

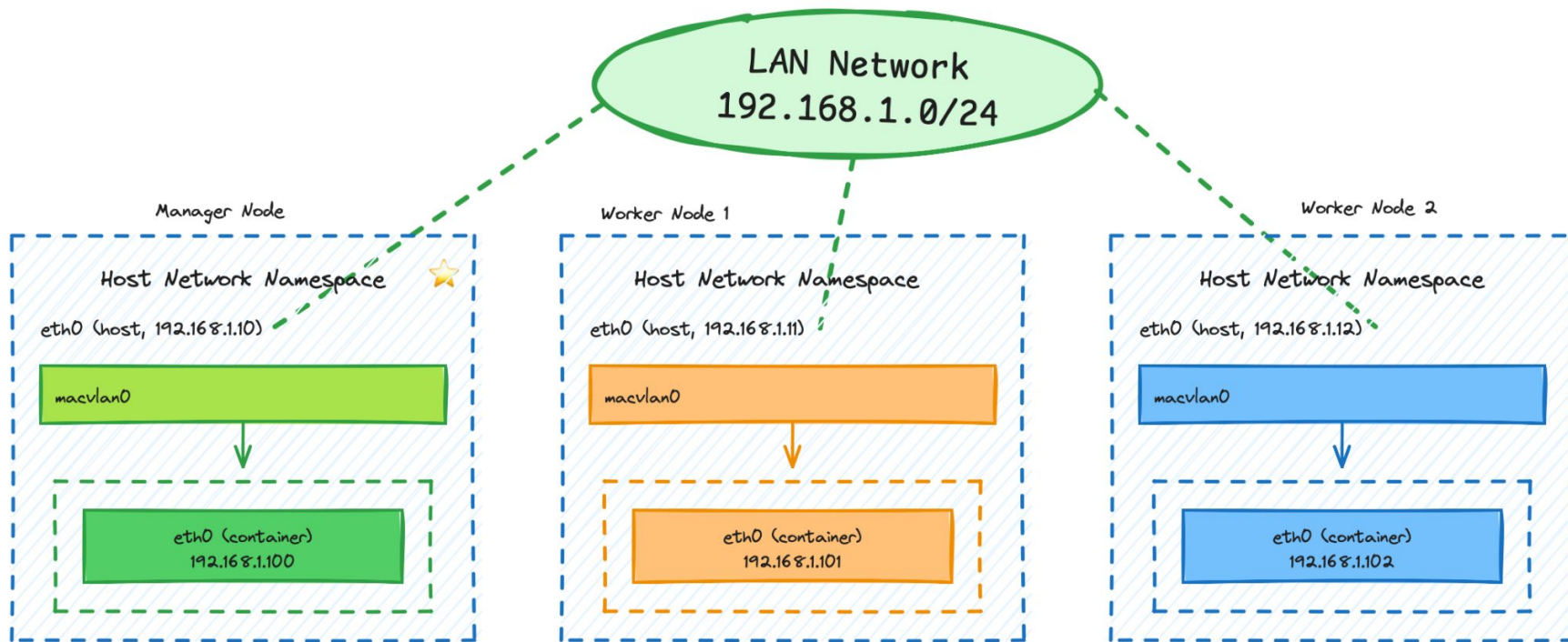
```
docker service create --name ingress-nginx --replicas 3 --publish  
published=8080,target=80 nginx:latest
```

```
docker service create --name int-nginx --network lab-overlay --replicas 1  
nginx:latest
```

Macvlan

- Контейнер получает собственный MAC-адрес
- IP из физической сети
- Видится как отдельное устройство

Macvlan



Note: Host ↔ Container Traffic Blocked by Default
(can be enabled by adding a macvlan interface on host with IP from same subnet)

Работа с macvlan

Создаём macvlan сеть

```
docker network create -d macvlan \
    --subnet=192.168.1.0/24 \
    --gateway=192.168.1.1 \
    -o parent=eth0 \
    macvlan_net
```

Запускаем контейнеры

```
docker run -d --name mac1 \
    --network macvlan_net \
    --ip 192.168.1.100 \
    alpine sleep infinity
```

Проверяем MAC-адреса (будут РАЗНЫЕ)

```
docker exec mac1 cat /sys/class/net/eth0/address
```

IPvlan

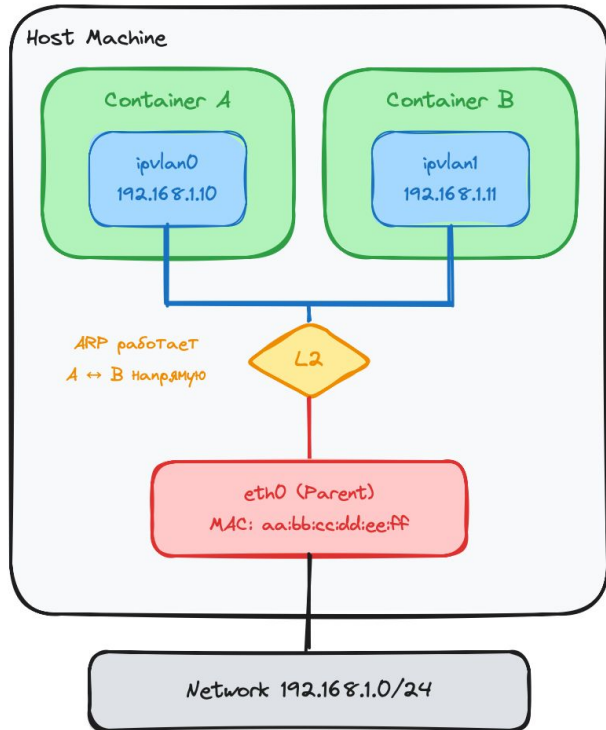
- Контейнеры используют **один MAC-адрес хоста**
- Полный контроль над **IPv4 и IPv6 адресацией**
- Поддерживает режимы **L2 и L3**
- Минимальная нагрузка на сеть по сравнению с Macvlan

Предназначен для интеграции с underlay-сетью

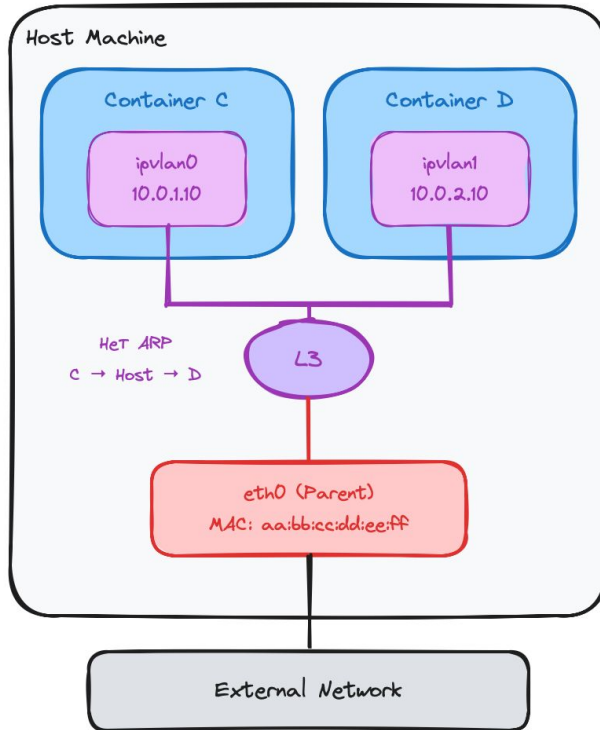
IPvlan

One MAC address, multiple IP addresses

IPvlan L2 Mode



IPvlan L3 Mode



Работа с ipvlan

Создаём ipvlan L2 сеть

```
docker network create -d ipvlan \  
  --subnet=192.168.1.0/24 \  
  --gateway=192.168.1.1 \  
  -o parent=eth0 \  
  -o ipvlan_mode=l2 ipvlan_l2_net
```

Создаём ОДНУ ipvlan L3 сеть с ДВУМЯ подсетями

```
docker network create -d ipvlan \  
  --subnet=10.0.1.0/24 \  
  --subnet=10.0.2.0/24 \  
  -o parent=eth0 \  
  -o ipvlan_mode=l3 ipvlan_l3_net
```

Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет

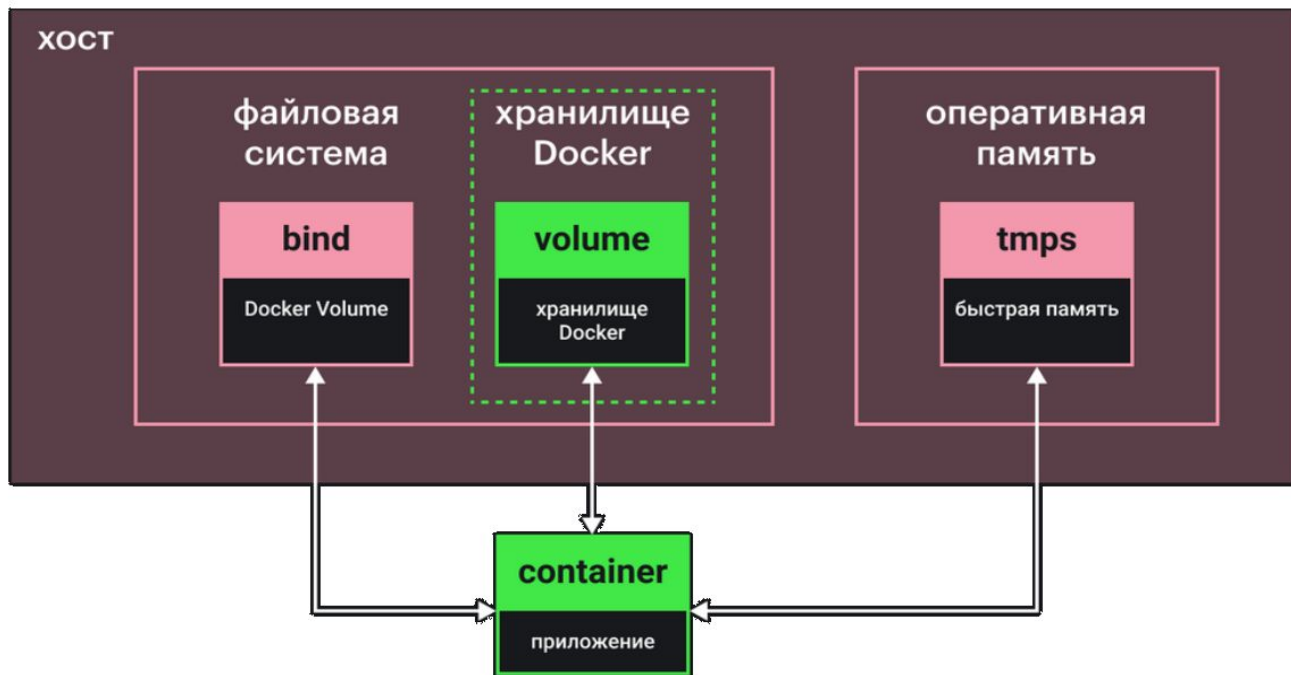


Хранение данных в Docker Тома (Volumes)

Томы (volumes) в Docker

- Используются для постоянного хранения данных (вместо r/w layer контейнера)
- **bind mount** — подключение директории из хостовой системы
- **docker volume** — том, управляемый docker (/var/lib/docker/volumes/)
- **tmpfs** — том в оперативной памяти

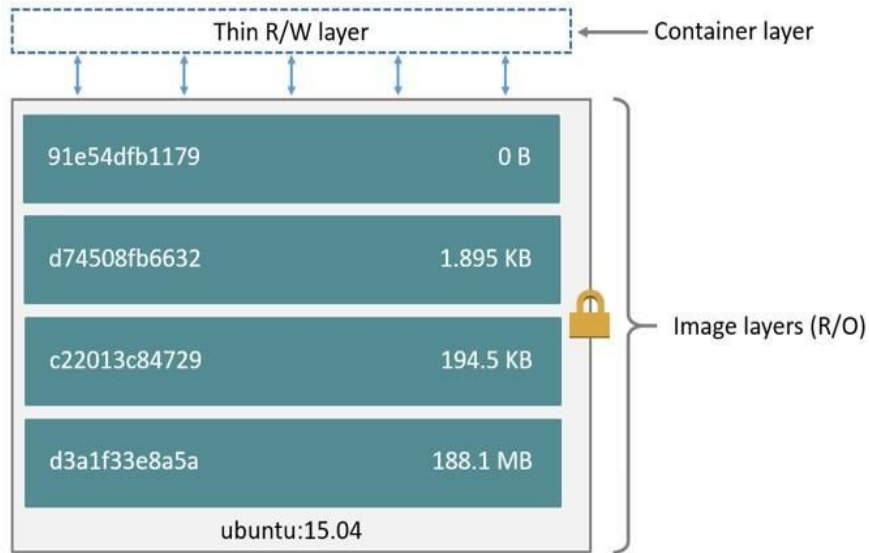
Хранение данных в Docker



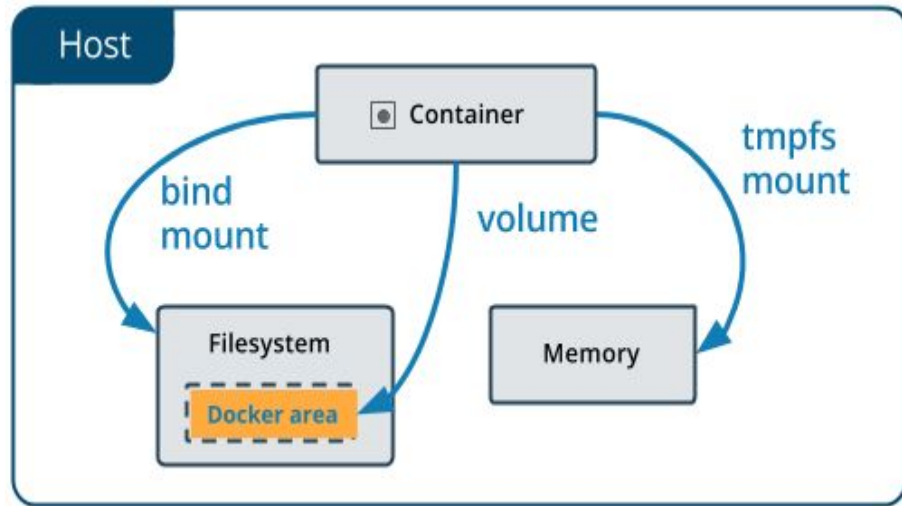
Какие данные стоит хранить постоянно?

- Конфигурационные файлы
- Скачиваемые для использования плагины
- Базы данных
- Настройки пользователей
- Данные для работы приложения

Постоянное хранение данных



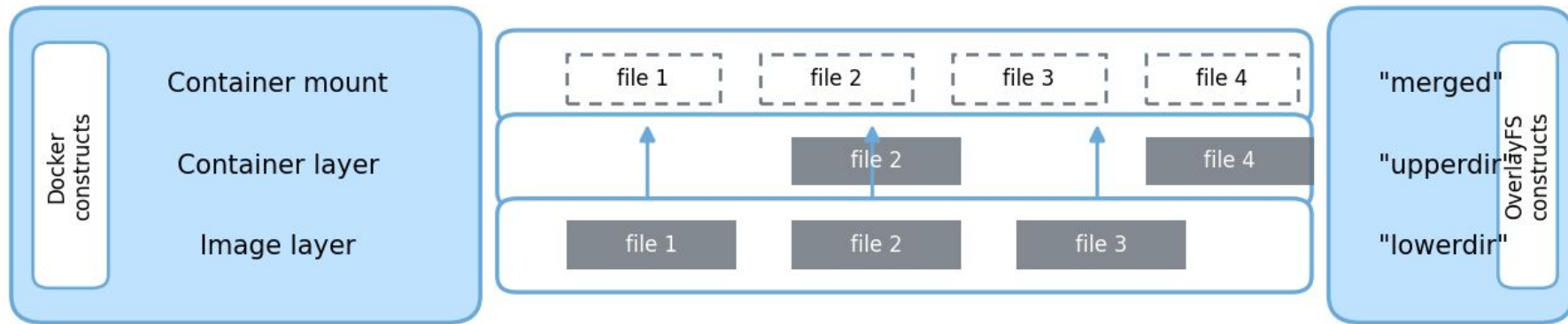
Container
(based on ubuntu:15.04 image)



Драйверы

overlay2	overlay2 является предпочтительным драйвером хранилища для всех поддерживаемых в настоящее время дистрибутивов Linux и не требует дополнительной настройки
overlay	Устаревший overlay драйвер использовался для ядер, которые не поддерживали функцию «multiple-lowerdir»
Btrfs, ZFS	В btrfs и zfs драйверах имеются дополнительные опции, такие как создание «снапшотов», но требуют большего обслуживания и настройки
aufs	Aufs Драйвер запоминающего устройства был предпочтительным драйвер для хранения Docker 18.06 и старше, при работе на Ubuntu 14.04 на ядре 3.13 , которая не имела поддержки overlay2. Однако в текущих версиях Ubuntu и Debian теперь есть поддержка драйвера overlay2, который является рекомендуемым драйвером
devicemapper	Использовался для производственных сред, но имел очень низкую производительность. Devicemapper был рекомендованным драйвером хранилища для CentOS и RHEL, поскольку их версия ядра не поддерживала overlay2. Однако в текущих версиях CentOS и RHEL есть поддержка overlay2

Overlay2



Containerd image store

- это часть движка containerd, которая отвечает за хранение, управление и извлечение файлов контейнерных образов на диске
- По умолчанию располагается в `/var/lib/containerd`.
- Он использует "снимки" (snapshotters) для управления слоями образов, что позволяет повторно использовать общие слои между разными образами, экономя место.
- containerd хранит данные как content-addressable blobs (sha256:...) в content store.
- Метаданные образов (теги/манифесты) — отдельно от самих blob'ов.

Работа с томами Docker

Связанные папки (bind mounts):

```
docker run -d -it --name devtest -v "$(pwd)"/target:/app node:1ts
```

```
docker run -d -it --name devtest --mount
```

```
type=bind,source="$(pwd)"/target,target=/app node:1ts
```

Тома (volumes)

```
docker run -d --name devtest -v my-vol:/app node:1ts
```

```
docker run -d --name devtest --mount source=my-vol,target=/app node:1ts
```

Из Dockerfile

```
FROM ubuntu:latest
```

```
RUN mkdir /data
```

```
WORKDIR /data
```

```
RUN echo "Hello from Volume" > test
```

```
VOLUME /data
```



Работа с томами Docker

```
# docker-compose.yml
services:
  app:
    image: nginx
    volumes:
      - ./html:/usr/share/nginx/html      # bind mount
      - app_data:/var/lib/data            # named volume

volumes:
  app_data:
```

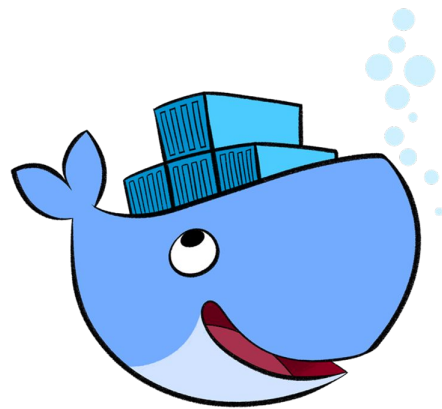
Хранение в оперативной памяти

C mount

```
docker run -d -it --name tmptest --mount \
type=tmpfs,destination=/app node:lts
```

Короткая форма

```
docker run -d -it --name tmptest --tmpfs /app node:lts
```



Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет

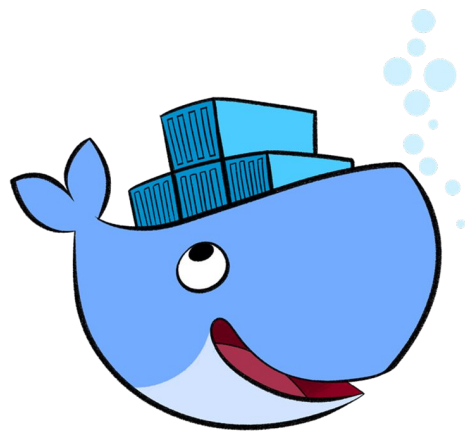


Установим свой docker registry



Что такое Docker registry?

- Это **репозиторий**, в котором хранятся **образы**. Когда разработчики создают приложения, они размещают свои образы в этих репозиториях, откуда их могут скачать другие люди.
- Есть публичные репозитории, например Docker Hub.
- Можно создать свой репозиторий, для использования внутри компании или команды.



Это всё Docker registry?

- Amazon Elastic Container Registry (ECR)
- Azure Container Registry (ACR)
- Docker Hub Container Registry
- GitHub Package Registry
- GitLab Container Registry
- Google Artifact Registry (GAR)
- **Harbor Container Registry**
- Red Hat Quay
- Sonatype Nexus Repository OSS



Запуск Harbor container registry

- Качаем offline-инсталлятор
- Получаем SSL-сертификат для домена
- Настраиваем harbor.yml
- Запускаем install.sh



<https://goharbor.io/docs/2.11.0/install-config/download-installer/>

Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет

Docker tips and tricks

- Можно удалять образы, не вводя полный хэш
`docker rmi 7c9 2f 5cad` удалит 3 образа
- Старайтесь именовать вольюмы чтобы потом их было легче найти
`docker run --name psql -d -p 5432:5432 -v psql:/var/lib/postgresql \`
`sameersbn/postgresql`
- `docker-compose down` позволяет подчистить всё за собой. (networks, containers)
`docker compose down -v --rm local`
- Удалить только остановленные контейнеры, не удаляя тома
`docker rm $(docker ps -aq)`
- Удалить все образы, которые в данный момент не используются в контейнерах
`docker rmi $(docker images -q -f "dangling=true")`
- Получить локальный IP контейнера
`docker inspect --format="{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}"`
- Полная очистка неиспользуемых контейнеров, сетей, образов и томов
`docker system prune -a -f --volumes`

Многоэтапные (multi-stage builds) сборки в Docker

`git clone https://github.com/codeship/go-hello-world.git`

```
FROM golang:1.8
WORKDIR
/go/src/github.com/codeship/go-hello-world
COPY hello-world.go .
RUN go build -o hello-world .

CMD ./hello-world
```

```
FROM golang:1.8 as build-stage
WORKDIR /go/src/github.com/codeship/go-hello-world
COPY hello-world.go .
RUN go build -o hello-world .

# starting second stage
FROM alpine:latest

# copy the binary from the `build-stage`
COPY --from=build-stage
/go/src/github.com/codeship/go-hello-world/hello-world /bin

CMD hello-world
```

Список материалов для изучения

1. [Docker install](#)
2. [Portainer](#)
3. [Harbor](#)
4. [Play with Docker](#)
5. [Docker Swarm](#)

Рефлексия

Цели вебинара

Проверка достижения целей

1. Работать с docker networks и docker volumes
2. Понять основные принципы работы Docker registry
3. Применить на практике навыки управления Docker

Вопросы для проверки

По пройденному материалу всего вебинара

1. Для чего нужны multi-stage builds?
2. Как отправить образ в docker registry?
Как получить образ из приватного репозитория?
3. Как создать свой собственный docker registry?
4. Зачем нужно монтировать директории с хоста?



Вопросы для проверки

Какой сетевой драйвер используется Docker по умолчанию?

- A. host
- B. none
- C. bridge
- D. overlay



Отправьте в чат верную букву

Вопросы для проверки

Какой сетевой драйвер используется Docker по умолчанию?

- A. host
- B. none
- C. bridge**
- D. overlay

Вопросы для проверки

Какой тип сетевого драйвера Docker полностью удаляет сетевую изоляцию между контейнером и хостом, используя сеть хоста напрямую?

- A. Bridge network
- B. Overlay network
- C. None network
- D. Host network



Отправьте в чат верную букву

Вопросы для проверки

Какой тип сетевого драйвера Docker полностью удаляет сетевую изоляцию между контейнером и хостом, используя сеть хоста напрямую?

- A. Bridge network
- B. Overlay network
- C. None network
- D. Host network**

Вопросы для проверки

Что такое Docker Volume?

- A. Управляемая Docker сущность для постоянного хранения данных
- B. Директория внутри контейнера
- C. Образ для хранения данных
- D. Архив с конфигурацией контейнера



Отправьте в чат верную букву

Вопросы для проверки

Что такое Docker Volume?

A. Управляемая Docker сущность для постоянного хранения данных

B. Директория внутри контейнера

C. Образ для хранения данных

D. Архив с конфигурацией контейнера

Вопросы для проверки

Какой драйвер хранилища является предпочтительным для всех поддерживаемых в настоящее время дистрибутивов Linux и не требует дополнительной настройки?

- A. Btrfs
- B. ZFS
- C. devicemapper
- D. overlay2



Отправьте в чат верную букву

Вопросы для проверки

Какой драйвер хранилища является предпочтительным для всех поддерживаемых в настоящее время дистрибутивов Linux и не требует дополнительной настройки?

- A. Btrfs
- B. ZFS
- C. devicemapper
- D. overlay2**

Вопросы для проверки

Для чего используется сетевой драйвер host?

- A. Для полной изоляции контейнера от сети
- B. Для связи контейнеров между разными хостами
- C. Для максимальной производительности без сетевой изоляции
- D. Для работы только с IPv6



Отправьте в чат верную букву

Вопросы для проверки

Для чего используется сетевой драйвер host?

- A. Для полной изоляции контейнера от сети
- B. Для связи контейнеров между разными хостами
- C. Для максимальной производительности без сетевой изоляции**
- D. Для работы только с IPv6

Рефлексия



Насколько тема была для вас сложной?



Как будете применять на практике то, что узнали на вебинаре?

Спасибо за внимание!

Приходите на следующие вебинары

Шиков Станислав

Начальник отдела автоматизации ООО Кодер

Об опыте:

9+ лет в DevOps

начинал с системного администрирования

преподаватель в OTUS на курсах DevOps, Linux, SRE и некоторых других, автор нескольких курсов

Эл. почта:

stenlav@mail.ru

